

NumPy

is the fundamental package for scientific computing in Python. Its primary offering is a powerful N-dimensional array object, along with a collection of fast, optimized functions for performing mathematical, logical, and statistical operations on these arrays

1.The Core: ndarray

At its heart is the ndarray, a container for homogeneous data (all elements of the same type) that allows for efficient, vectorized operations .

2. Import Convention

python

```
import numpy as np
```

3. Creating Arrays

Function	Description	Example
<code>np.array([1, 2, 3])</code>	Create an array from a Python list .	<code>array([1, 2, 3])</code>
<code>np.zeros((3, 4))</code>	Create an array filled with zeros (default dtype is float64) .	<code>array([[0., 0., 0., 0.], ...])</code>
<code>np.ones((2, 3))</code>	Create an array filled with ones .	<code>array([[1., 1., 1.], [1., 1., 1.]])</code>
<code>np.eye(5)</code>	Create a 5x5 identity matrix .	<code>array([[1., 0., ..., 0.], [0., 1., ..., 0.]])</code>
<code>np.arange(0, 10, 2)</code>	Create an array with a fixed step size (start, stop, step) .	<code>array([0, 2, 4, 6, 8])</code>
<code>np.linspace(0, 100, 5)</code>	Create an array with a specified number of evenly spaced values .	<code>array([0., 25., 50., 75., 100.])</code>
<code>np.random.rand(2, 3)</code>	Create an array of random floats between 0 and 1 .	<code>array([[0.1, 0.7, 0.3], [0.5, 0.9, 0.2]])</code>
<code>np.full((2, 3), 7)</code>	Create a constant array .	<code>array([[7, 7, 7], [7, 7, 7]])</code>

4. Inspecting Array Properties

Attribute	Description	Example
<code>arr.shape</code>	Dimensions of the array (rows, columns) .	(3, 4) for a 3x4 array
<code>arr.size</code>	Total number of elements .	12
<code>arr.dtype</code>	Data type of the elements (e.g., int64, float64) .	<code>dtype('int64')</code>
<code>arr.ndim</code>	Number of dimensions .	2

5. Array Manipulation & Indexing

Operation	Description	Example
Indexing	Access elements (0-indexed) .	<code>arr[0, 0]</code>
Slicing	Extract sub-arrays using start:stop:step .	<code>arr[1:3, :]</code> (rows 1 & 2, all columns)
Boolean Masking	Filter an array using a condition of the same shape .	<code>arr[arr > 5]</code>
Fancy Indexing	Index using integer arrays .	<code>arr[[0, 2, 4]]</code>
<code>arr.T</code>	Transpose the array .	<code>arr.T</code>
<code>arr.reshape(2, 6)</code>	Change the shape of an array . Use -1 to infer one dimension.	<code>arr.reshape(-1, 1)</code>
<code>arr.flatten()</code>	Return a copy of the array collapsed into one dimension .	<code>arr.flatten()</code>
<code>np.concatenate()</code>	Join arrays along an existing axis .	<code>np.concatenate([a, b], axis=0)</code>

6. Key Mathematical Operations

NumPy operates on an element-wise basis. This is known as vectorization and is much faster than Python loops .

Category	Functions	Example
Arithmetic	np.add, np.subtract, np.multiply, np.divide, np.power, np.mod .	arr1 + arr2, arr ** 2
Trigonometric	np.sin, np.cos, np.tan, np.arcsin etc. .	np.sin(arr)
Exponents & Logs	np.exp, np.log, np.log10 .	np.exp(arr)
Statistical (Reduction)	np.sum, np.mean, np.std, np.var, np.min, np.max, np.median . Use the axis parameter to specify dimensions.	np.mean(arr, axis=0)

7. Key Concepts

- **Broadcasting:** A powerful set of rules that allows NumPy to perform arithmetic operations on arrays of different shapes. The smaller array is "stretched" to match the larger one without making copies of the data .
- **Vectorization:** Replacing explicit Python loops with array operations. This leverages C-level optimization for significantly faster computation .
- **axis Parameter:** Many functions accept an axis parameter to specify which dimension to operate on. For example, axis=0 operates column-wise, while axis=1 operates row-wise on a 2D array .